



PDF Download
3712256.3726396.pdf
27 March 2026
Total Citations: 0
Total Downloads: 956

 Latest updates: <https://dl.acm.org/doi/10.1145/3712256.3726396>

RESEARCH-ARTICLE

Transformers as Surrogate Models for Genetic Programming in AutoML Tasks

MATHEUS CÂNDIDO TEIXEIRA, Federal Institute of Education, Science and Technology of Mato Grosso, Cuiaba, MT, Brazil

GISELE LOBO PAPPA, Federal University of Minas Gerais, Belo Horizonte, MG, Brazil

Open Access Support provided by:

Federal Institute of Education, Science and Technology of Mato Grosso

Federal University of Minas Gerais

Published: 13 July 2025

Citation in BibTeX format

GECCO '25: Genetic and Evolutionary
Computation Conference
July 14 - 18, 2025
Malaga, Spain

Conference Sponsors:
SIGEVO

Transformers as Surrogate Models for Genetic Programming in AutoML Tasks

Matheus Cândido Teixeira
matheus.candido@ifmt.edu.br
Instituto Federal de Mato Grosso
Cuiabá, Brazil

Gisele Lobo Pappa
glpappa@dcc.ufmg.br
Universidade Federal de Minas Gerais
Belo Horizonte, Brazil

Abstract

In applications where the fitness function has a high computational cost, one of the main drawbacks of Evolutionary Algorithms when compared to other search methods is a prohibitive computational cost. The use of surrogates as proxies for fitness function calculation to alleviate this problem is not new, but addressing the problem as a binary relation learning, i.e., evaluating if one individual is better or worse than another without estimating the actual value of the fitness, is a recent trend.

This paper proposes a transformer-encoder as a surrogate to evaluate pairs of solutions and determine their relationship, i.e., which one is better/worse than the other. We experimented the model in the context of AutoML, which seeks to find the best combination of algorithms for a classification problem. To optimize the pipeline, we can use a genetic programming, but the cost of evaluating each individual is generally expensive.

We trained the encoder with several parameters and compared its performance against traditional GP – evaluating fitness at each generation. Results confirm using the encoder as a surrogate does not degrade the fitness values of the evolved population of ML pipelines and can even improve it in some cases (up to 285 times faster).

Keywords

transformers, fitness surrogates, AutoML

ACM Reference Format:

Matheus Cândido Teixeira and Gisele Lobo Pappa. 2025. Transformers as Surrogate Models for Genetic Programming in AutoML Tasks. In *Genetic and Evolutionary Computation Conference (GECCO '25)*, July 14–18, 2025, Malaga, Spain. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3712256.3726396>

1 Introduction

The powerful representation provided by transformer models [25] and their outstanding results in various problems, particularly in natural language processing with large language models (LLMs), have led to widespread adoption by numerous research communities [14, 27]. This includes the field of evolutionary computing

[15], where recent advancements have explored the integration of transformers into the evolutionary process.

In recent years, Transformer models have been used within evolutionary algorithms (EAs) in multiple ways: for generating populations, replacing genetic operators, or serving as surrogate models to estimate fitness values [15]. The application of these models to fitness estimation is particularly valuable, as it addresses a major drawback of evolutionary algorithms compared to other optimization methods – their prohibitive computational cost.

Traditional surrogate models tackle fitness estimation as a regression problem, attempting to predict an individual's precise fitness score. However, regression models inevitably introduce errors, which can propagate through generations and impact the evolutionary process. The approach introduced here leverages the concept of binary relation learning [10], where the surrogate model determines whether one individual is superior to another in terms of fitness. Formally, given two individuals p_i, p_j , the model learns the relation $f(p_i) \geq f(p_j)$, where $f : \mathcal{X} \rightarrow \mathbb{R}$ represents the fitness function. This approach reduces the impact of small estimation errors since a single misclassification does not drastically alter the evolutionary trajectory, and speeds up the process of fitness evaluation by up to 235 times.

To the best of our knowledge, the only work in the literature that has used a similar approach to create a surrogate for classical black-box optimization problems is [11]. The authors integrated a decoder model into an evolutionary algorithm. Given a prompt, the decoder returns the fitness relationship between two individuals. The approach proposed in this paper, in contrast, uses an encoder to output whether individual p_i is better or worse than p_j .

Recall that a key feature of the Transformer architecture is the attention mechanism, which enables models to “interpret” a neural network input (token) based on its surrounding context. Over time, several works have leveraged different components of the Transformer model: the decoder for text generation tasks and the encoder for classification tasks. The main difference between encoder and decoder models is what the attention layers can access in each stage of the learning process. In the encoder, the attention layers can access all the words given as input (i.e., the initial sentence), and are well-known as auto-encoding models. In the decoder, at each phase of the learning process, for a given word, the attention layers can only access the words positioned before it in the input (i.e., sentence), and are often called auto-regressive. Given these characteristics, encoders are best suited for tasks where a full-sentence understanding is needed, which we believe is the case for fitness comparisons, where more context than the previous token is necessary.

The proposed encoder-based methodology is integrated with a grammar-based genetic programming (GP) algorithm [5]. We

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GECCO '25, July 14–18, 2025, Malaga, Spain

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1465-8/2025/07

<https://doi.org/10.1145/3712256.3726396>

believe this approach is generic enough to be applied to many different problems, from symbolic regression to Automated Machine Learning (AutoML). In this paper, however, we focus on AutoML because it is more cost-intensive and more applicable for a practical situation. AutoML basically consists of generating and evaluating machine learning pipelines [18].

Evolutionary algorithms have been previously used to search AutoML spaces [26]. The particular search space we explore in this paper has a high locality, i.e., similar pipelines tend to have similar fitness values. This property suggests that transformers, with their ability to capture relationships between elements in sequences, can effectively estimate which AutoML pipeline is superior based on its algorithmic components and hyperparameters.

The results indicate the model's performance can be better or preserve the values of fitness of traditional methods while reducing optimization time by up to 235 times. Several experiments showed that the overall fitness of the population tends to be superior using the model proposed in this work as a surrogate.

This paper is organized as follows. Section 2 reviews related works and their contributions to our research. Section 3 describes the proposed method and how it can be integrated with a Grammar-based Genetic Programming (GGP). Section 4 details our experimental setup and objectives. Section 4 presents our results and their analysis. Finally, Section 5 concludes the paper and suggests future research directions.

2 Related Work

Recent advancements in the integration of Large Language Models (LLMs) into Machine Learning (ML) and optimization workflows have demonstrated significant improvements in automation, adaptability, and efficiency. It was not different in evolutionary computation.

Liu et al. [15] used LLMs to implement evolutionary operators, designing prompts for generating populations, selecting individuals, performing mutations, and recombining solutions. Their approach aligns with Hao et al. [11], who used LLMs as surrogate models to estimate individual performance in an optimization process. By providing training data on individual fitness, LLMs predict evaluations, refining the selection of individuals added to the population. This strategy enhances efficiency in solving NP-hard problems, such as the Traveling Salesman Problem (TSP).

Gu et al. [9] surveyed the broader integration of LLMs in ML workflows, emphasizing their role in automating key stages such as data preprocessing, feature engineering, model selection, hyperparameter tuning, and evaluation. Their findings highlight the adaptability and automation that LLMs bring, which is also evident in AutoML-GPT by Zhang et al. [31]. AutoML-GPT leverages structured descriptions (data and model cards) to create prompts for LLMs, automating ML pipeline tasks from data processing to hyperparameter tuning, demonstrating adaptability across diverse domains like computer vision and NLP.

Xiao et al. [29] introduced Verbalized Machine Learning (VML), which further explores LLM-based automation by integrating natural language as a parameter space for ML models. Unlike conventional numerical optimization, VML encodes human-interpretable

text prompts, allowing LLMs to function as learners and optimizers. This aligns with Xu et al. [30], who developed Text-to-ML, a framework combining LLMs with AutoML to generate complete ML workflows from textual task descriptions. Text-to-ML divides workflows into modular components—data preparation, modeling, and post-processing—ensuring seamless integration through AutoML-based optimization. Experiments showed that Text-to-ML outperformed existing methods in 10 out of 12 tasks, particularly excelling in deep learning applications beyond LLM training data.

This paper takes advantage of LLMs to improve the time of fitness calculations using a transformer encoder as a surrogate for black-box optimization problems. It differs from the work of [15] that uses a decoder to perform this task, as we believe an encoder can better leverage the structure of ML pipelines. It also helps to solve a computationally intensive task, making its benefits beyond those expected.

3 Methodology

In evolutionary algorithms (EAs), LLMs are integrated into the EA framework in a modular way, i.e., the original method where LLMs are introduced makes a call for an LLM, giving a prompt (in the case of a decoder) or an example. This section first introduces the original GP system, followed by details on the LLM and its integration with the traditional framework.

3.1 AutoML

The problem of AutoML can be formally defined as a generalization of the Combined Algorithm Selection and Hyperparameter optimization (CASH) problem [8]. In its original definition, given a set $\mathcal{A} = \{A^{(1)}, A^{(2)}, \dots, A^{(k)}\}$ of learning algorithms, where each algorithm $A^{(j)}$ has a hyperparameter space $\Lambda^{(j)} = \{\lambda^{(1)}, \dots, \lambda^{(S)}\}$, defined from the full set of algorithm's hyper-parameters Ω , the CASH problem is defined as in Eq. 1¹.

$$A_{\lambda^*}^* = \underset{A^{(j)} \in \mathcal{A}, \lambda \in \Lambda^{(j)}}{\operatorname{argmax}} \frac{1}{k} \sum_{i=1}^k \mathcal{F}(A_{\lambda}^{(j)}, \mathcal{D}_{train}^{(i)}, \mathcal{D}_{valid}^{(i)}) \quad (1)$$

where $\mathcal{F}(A_{\lambda}^{(j)}, \mathcal{D}_{train}^{(i)}, \mathcal{D}_{valid}^{(i)})$ is the gain achieved when a learning algorithm A , with hyperparameters Λ , is trained and validated on disjoint training and validation sets $\mathcal{D}_{train}^{(i)}$ and $\mathcal{D}_{valid}^{(i)}$, respectively, on each partition $1 \leq i \leq k$ of a k -fold cross-validation procedure.

A generalization can be made if we replace $\mathcal{A}$ by a set of pipelines $\mathcal{P} = \{P^{(1)}, \dots, P^{(V)}\}$, which includes a subset of algorithms from $\mathcal{A}$ and their respective set of hyperparameters $\Gamma^{(i)} = \{\Lambda^{(1)}, \dots, \Lambda^{(S)}\}$, represented by the full set Ψ , as defined in Eq. 2

$$P_{\Gamma^*}^* = \underset{\mathcal{P}^{(i)} \subseteq \mathcal{P}, \Gamma^{(i)} \subseteq \Psi}{\operatorname{argmax}} \frac{1}{K} \cdot \sum_{j=1}^K \mathcal{F}(P^{(i)}, \Gamma^{(i)}, \mathcal{D}_{train}^{(j)}, \mathcal{D}_{valid}^{(j)}) \quad (2)$$

In this case, instead of looking for an algorithm, we look for one or more combination of algorithms that maximize $\mathcal{F}$.

¹The original definition casts the problem as a minimization one. Here we replace the loss function by a gain function.

3.2 Auto-ML Grammar-based Genetic Programming

AutoML methods are designed to build and optimize machine learning pipelines — a set of operations that includes data preprocessing, classification, and postprocessing — for a classification or regression task [12]. Normally, creating a machine learning model requires a lot of manual work, like cleaning and preparing data, selecting the right algorithm, tuning its parameters, and testing different configurations to find the best one. AutoML takes care of most of these tasks automatically.

This section introduces a grammar-based GP (GGP) [17] generated for this purpose. The idea of the algorithm is to generate pipelines based on a grammar, which guarantees the individuals generated are valid. The grammar comes from previous work [20], and includes a set of 38 production rules². In terms of preprocessing, it includes algorithms that deal with feature scaling and dimensionality reduction, such as PCA. It is also possible for a pipeline to ignore preprocessing steps. In terms of classification methods, there are five possible options: Logistic Regression, Multilayer Perceptron, K-Nearest Neighbors (KNN), Random Forest, and Ada Boost. The number of hyperparameters varies from one classification algorithm to another, going from two (Ada Boost) to 7 (Random Forest).

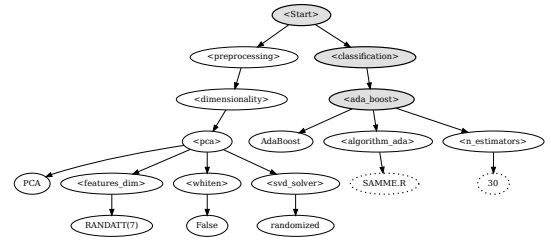
The main reason we chose to work with this search space is that it was designed so that an exhaustive evaluation could be performed, so that the performance of all possible solutions (ML pipelines) would be known. This way, we can measure how close the model got to the global optima.

In GGP algorithms, each individual is derived from the grammar following a set of production rules. Figure 1 shows two individuals (ML pipelines) generated from the grammar. Note that both root nodes are labeled “<Start>”, which is a start symbol of the grammar, present in all trees. While the preprocessing and classification algorithms are identical in the pipelines in both figures, the hyperparameters are different.

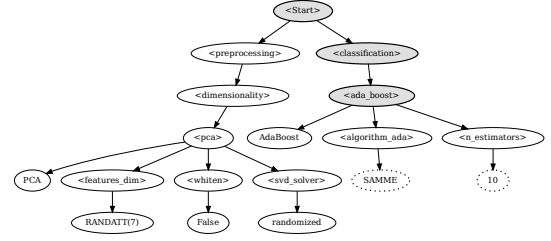
Fitness. To calculate the fitness of an individual, we generate the code of the ML pipeline it represents using the SciKit Learn Python library [19]. The code is then executed in the target dataset, and the weighted average F-measure is calculated [28]. This metric is given by the harmonic mean of precision and recall. As some datasets have several classes, the one-vs-all strategy was employed when calculating this metric.

When evaluating the fitness, a maximum computational budget of 120 seconds (for training and testing the pipeline) was defined, and solutions that exceeded this limit received fitness 0. This was the naive way we found to deal with the trade-off between the computational cost versus the accuracy of the solutions.

Selection. We use the tournament selection to decide which individuals should compete with each other. We used tournament size 5 because it was the value that resulted in the best fitness for GGP. This means that we select the top 5 individuals with the highest fitness from the population to generate new population of individuals through mutation and crossover.



(a) Pipeline #1



(b) Pipeline #2

Figure 1: Examples of ML pipelines.

Mutation. The mutation operator works as follows. If an individual undergoes mutation, a node in the tree is selected with a probability proportional to its distance from the root. This is important because nodes closer to the root are more likely to introduce big changes into the pipelines. For example, if a tree has up to L levels, the probability of selecting a node at a specific level is given by:

$$p_L = \frac{1}{L}$$

The probability of selecting a node n at level $l \in [1, L]$, in turn, is:

$$p(l) = \frac{1}{N(l)}$$

where $N(l)$ is the number of nodes at level l . Hence, the probability of an arbitrary node n being selected is:

$$p(n) = p_L \times p(l) = \frac{1}{L \times N(l)}$$

After selecting the node, a random subtree – following the productions of the grammar – is generated.

Crossover. The crossover operator randomly selects a common node between the two trees involved in the operation and swaps the subtrees from that node.

For example, consider that Pipelines #1 and #2 from Figure 1 are selected for crossover. Only the common nodes can be selected. Let us assume the node “<ada_boost>” is selected, as it is present in both trees. The result would be two new trees, where the first one would be identical to Pipeline #1 but with the subtree originating from the selected node equal to that of the second Pipeline #2. The same process is applied to Pipeline #2.

²The complete grammar in BNF is available at <https://bit.ly/38F0o3U>

3.3 Encoder-based surrogate model

Considering the grammar-GP introduced previously, we modify its selection process to estimate which one is best given a pair of selected individuals.

For that to happen, we first train an encoder model to learn when one individual is superior or inferior to another in terms of fitness. This section details the proposed model and how it was trained.

Model Architecture and Training. The Transformer architecture [25] became the state of the art after its publication in 2017. This model stood out for handling sequential data in a way that allowed training parallelization and for the presence of the attention mechanism, which enabled the model to learn more complex relationships between tokens. In the following years, other papers proposed models derived from the original Transformer architecture. Some architectures were based solely on the Encoder [7, 16], while others focused exclusively on the Decoder [2].

Figure 2 shows the proposed model structure. It receives as input two pipelines, structured in the format

$[CLS] < pipeline > [SEP] < pipeline > [SEP]$

similar to the structure used by Devlin et al. [7] for text classification. The [CLS] token is generally used for sentence classification and the [SEP] token is used as a sentence separator in the context of BERT. In this paper, [SEP] is used to separate ML pipelines. As the pipelines need to be represented by a sequence, we read the leaf nodes of the derivation trees, which contain only terminal nodes, as illustrated in Figure 2. We make this decision because non-terminal nodes are the same for all trees and therefore do not provide information for the model to decide which pipeline is best. Furthermore, removing redundant information helps to reduce the dimensionality of the model, which results in a smaller and faster model.

The text is then passed through the tokenizer, transforming each token into an index (an integer). We used the WordPiece tokenizer [22] trained on ML pipelines generated for the wine-quality-red dataset. We mention this dataset because the grammar has one terminal that is dataset-dependent: the number of attributes. It is used to optimize the percentage of attributes that should be selected. Hence, the constants used to define these hyperparameters may not be present in other datasets. There are other approaches to deal with this problem, but we kept this strategy to make the problem more closely related to a real-world scenario, where there may always be unknown tokens. The vocabulary has 301 tokens (including the five manually added special tokens from WordPiece).

Following this process, the list of indices goes through the embedding, which associates a vector of real numbers to the index generated by the tokenizer. Next, the Positional Embedding layer adds information so that the encoder can handle the notion of the token's position in the sequence (in the sentence). Finally, the encoder receives the data along with the padding mask, used to make sentences have the same size and tells the model which positions it should consider during the analysis.

The Encoder generates a real vector for each token in the sequence. However, we are only interested in the "[CLS]" token, as it carries the information of the entire sequence, meaning it essentially summarizes the sequence [23]. Therefore, we select only

the first token of the sequence and use a linear layer to adjust the encoder's output to only two dimensions. If the value of the first dimension is greater than the value of the second, we consider that the first pipeline is better than the second and vice versa.

In the output, the first position indicates that the first pipeline is better than the second, and the second position that the second solution is better than the first. We consider that if the value in the first position is greater than the value in the second position, then the first pipeline is better than the second; and vice versa.

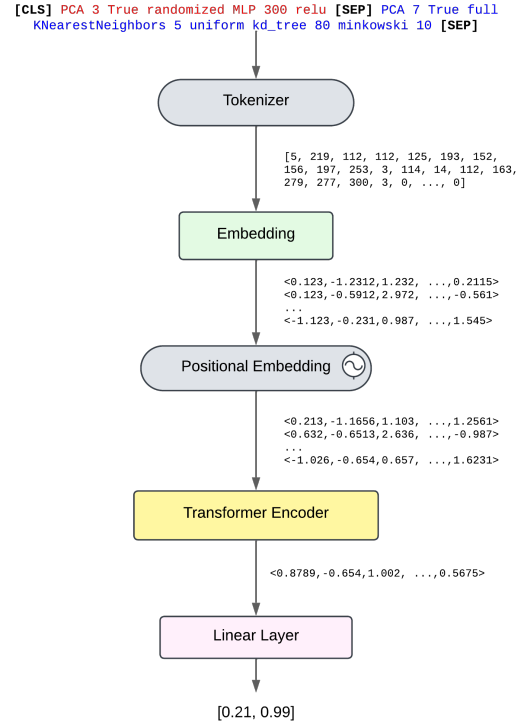


Figure 2: Model Architecture.

Regarding the hyperparameters of the model, they have not been extensively optimized. In empirical tests, we found that the smaller the batch size, the more unstable the loss. Hence, we set the batch size as 192 sequences (the highest value supported by our computing resources). The same happened with the learning rate: the higher it is, the more unstable the loss. We set its value at 0.0001. We have also checked the effect of the Encoder's dimensionality, running experiments with 64, 128 and 256 dimensions. The best results (lowest loss) were obtained with 128 dimensions. We also tested the architecture with 4 and 8 attention layers and found that 4 achieved a lower loss.

Finally, we tested several combinations of layers before defining the linear layer of the architecture presented in Figure 2. We tested the effect of using Batch Normalization [6], Dropout [4], and multiple linear layers with non-linear activation function (ELU [21]), but none of them presented a significant gain in the final result of

the model, and that is why we opted for a simple linear transformation, i.e., just one linear layer is used to project the Encoder output (d_model) to the desired dimension, in this case, 2.

We also tested the Stochastic Gradient Descent (SGD) optimization algorithm [3], but its performance was very modest compared to the Adam optimizer [32]. Furthermore, as discussed above, we used a constant learning rate, but we believe that it may be beneficial to use some kind of decay in the future.

Training Data. As mentioned earlier, we use a grammar to ensure that the generated individuals represent valid ML pipelines. The grammar allows for the representation of 69,960 different pipelines. This number may be relatively small, but it is sufficient to analyze whether using the Encoder as a surrogate is appropriate.

The goal is to train the encoder to detect the best ML pipeline among two options. Therefore, we generate pairs with the total number of pipelines. In total, there are 4,894,331,640 possible combinations (if the order matters) or 2,447,095,861 combinations (if the order does not matter).

To build the dataset,³ we randomly selected the pipelines with uniform distribution. We ensured that the same pair was not used more than once (regardless of order) and that there was no leakage of a particular pair into the test data.

3.4 Grammar-GP and Encoder Integration

Before integrating the LLM with the GP, we train the model, as previously described. Having the model ready to perform inferences, we modify the individual selection process of the Grammar-GP (GGP). After individuals are selected, they are compared in pairs using the model, and from these comparisons, a ranking is generated, returning the best individual. We refer to this method, which integrates the surrogate and GGP, as GGPEnc.

4 Computational Results

Experiments were performed in two phases. First, we evaluated whether the model was capable of learning which pipeline was the best among a pair of pipelines. Next, we analyzed the impact of integrating the LLM surrogate into the Grammar-GP. These results are discussed after we set the experimental setup.

4.1 Experimental setup

Table 1 lists the parameters used to train the model and run the GP. For a more detailed view, we recommend visiting our public repository on GitHub.

For the GGP, we tested the surrogate performance in 10 datasets from UCI [13], Kaggle [1] and OpenML [24]. The characterization of the datasets is presented in Table 2. The 'Size' column indicates the number of samples; the 'Features' column indicates the dimensionality of the dataset; the 'Classes' column indicates the number of distinct values present in the target feature and the 'Max F1' column indicates the highest F1-score (optimal value) obtained by the pipelines generated from the grammar.

However, it is important to emphasize that we did not train the encoder for each dataset individually. Instead, we select a dataset

³Be aware that dataset appears in two different contexts in this paper. The first is the dataset we are building a ML pipeline to. The second is the dataset built to train the encoder.

Table 1: List of parameters used in each algorithm.

Parameter	Value	Method
population_size	30	GP
generations	30	
prob_crossover	0.8	
prob_mutate	0.1	
d_model	128	Encoder
nhead	8	
dim_feedforward	128	
num_layers	4	
Learning rate	0.0001	Training
Optimizer	Adam	
Loss	Cross Entropy	
Train batch size	192	
Test batch size	192	

Table 2: Datasets used to train Machine Learning pipelines

Dataset	Size	Features	Classes	Max F1
abalone	4,177	8	28	0.28
bank	11,162	16	2	0.84
car-evaluation	1,728	6	4	0.94
diabetes	768	8	2	0.79
dry-bean	13,611	16	7	0.93
fire	17,442	6	2	0.95
ml-prove	6,118	51	6	0.45
texture	5,500	40	11	1
vehicle	846	18	4	0.79
wine-quality-red	1,599	11	6	0.64

at random, in this case *wine-quality-red*, — because there is no evidence that one could better than others — and trained the model with pairs of ML pipelines evaluated in this specific dataset. Next, we fine-tuned this initial model using data from the other 9 datasets.

The model built with *wine-quality-red* was trained for 100 epochs. For the other datasets, 30 epochs were run for fine-tuning. These parameters were chosen through manual inspection. We gradually increased the number of epochs and observed that there was no overfit even with 100 epochs. In the case of fine tuning, we observed that with 30 generations, the performance achieved was already close to the performance of training the model from scratch up to 100 generations.

Regarding the training of GGPEnc, it was necessary to train 54,692 pipelines on the *wine-quality-red* dataset to build the dataset to train GGPEnc and it took 3,176 minutes to evaluate the fitness of each pipeline. This number of pipelines corresponds to 78% of the total pipelines that can be represented by the grammar. Although the cost is high given the total number of pipelines in the search space, in problems with larger spaces, GGPEnc can be more efficient than GGP because the inference time is constant and, through transfer learning, the cost to optimize other datasets is lower.

4.2 Encoder Model Evaluation

We start by looking at the impact that different numbers of training samples would have on the model performance. Figures 3 and 4 show the loss during model training and test. Dataset size represents the number of examples in the training and test sets. Training always uses 70% of data while the test set includes the remaining 30% (i.e., in the case of 60,000, the largest dataset, 42,000 are used for training and the other 18,000 for test).

Figure 3 shows experiments with the *wine-quality-red* dataset, where the model was trained for 100 epochs. Note that the model trained with 1,000 samples presented overfitting from epoch 40. The higher the number of examples, the lower the loss becomes, and the closer the curves from different dataset sizes become.

Figure 4 shows examples of the loss of the models that received the model generated by *wine-quality-red* and fine-tuned it for the other datasets for 30 epochs. Notice that here the loss is more stable even for 1,000 examples. However, in these datasets, we stopped training before the model overfitted for the *wine-quality-red* dataset. As expected, in all datasets, at 10,000 the loss decreased monotonically with no signs of overfitting.

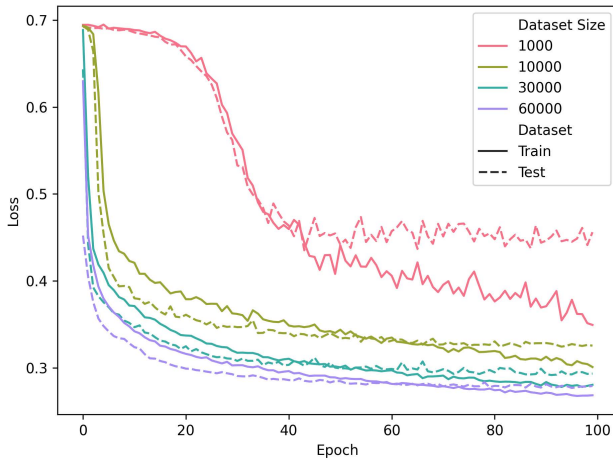


Figure 3: Loss behavior when training the encoder with the *wine-quality-red* dataset as a function of dataset size. Solid lines represent the loss in the training dataset and the dashed lines represent the loss in the test dataset. The color indicates the dataset size.

For the next experiments, we reported results with the model trained with 60,000 examples, although the previous result shows we can find a good trade-off between the number of examples and surrogate accuracy. Table 3 reports the values of accuracy, precision, recall and f-measure for the 10 datasets analyzed. The model should achieve near-optimal performance in each metric to ensure it reflects results similar to those obtained when directly evaluating fitness.

All results were above 0.9, except for the recall on the *texture* dataset, although the value was still relatively close. It is reasonable to assume, based on the results in the table, that the model “learned”

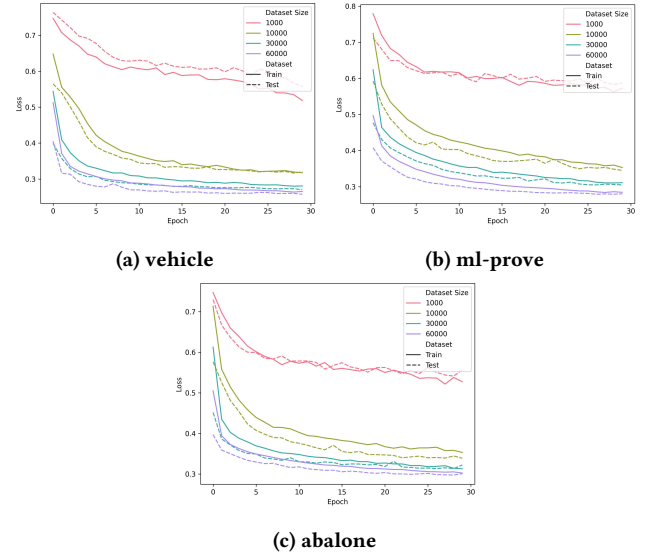


Figure 4: Loss behavior when training the encoder as a function of dataset size.

to identify which ML model is most effective for the analyzed dataset.

Table 3: Performance of the encoder model after training. The metrics were evaluated on test data.

Dataset	Accuracy	Precision	Recall	F1
abalone	0.9420	0.9384	0.9449	0.9417
bank	0.9678	0.9703	0.9654	0.9678
car-evaluation	0.9538	0.9496	0.9594	0.9545
diabetes	0.9393	0.9406	0.9403	0.9404
dry-bean	0.9702	0.9747	0.9648	0.9697
fire	0.9664	0.9673	0.9653	0.9663
ml-prove	0.9546	0.9542	0.9561	0.9551
texture	0.9027	0.9100	0.8947	0.9022
vehicle	0.9604	0.9684	0.9526	0.9604
wine-quality-red	0.9509	0.9434	0.9611	0.9522

As an alternative evaluation method, we measured the correlation between the fitness of the population ranking generated by the model and the ranking of the real fitness. Considering an ideal model, a population P ordered by the model would result in a ranking of the form $\text{Model}(P) = \langle I_1, I_2, \dots, I_n \rangle$, where $\text{fitness}(j_k) \geq \text{fitness}(j_{k+1})$. Therefore, the correlation between the fitness of the population ordered by the model and the fitness of the population sorted in descending order should be equal to 1, that is,

$$\text{Spearman}(\text{Fitness}(\text{Model}(P)), \text{Sorted}(\text{Fitness}(P))) = 1$$

We used this experiment to evaluate the quality of the model’s ranking.

In this evaluation process, we randomly selected 20 pipelines and ranked them based on the model’s output. Next, we measured the

correlation between the pipelines' fitness and their ranked fitness. The results are shown in Table 4. An ideal model should yield a correlation of 1, as the pipelines would be ranked perfectly according to their fitness. As observed, Spearman's correlation (which does not assume linear correlation) reached values close to 1, and all p-values are significant with a 95% confidence level. Therefore, we can conclude that there is a high correlation between the fitness values, meaning the model is capable of ranking the pipelines according to their fitness without requiring their evaluation.

Table 4: Spearman correlation between rankings generated by the surrogate model and rankings of the real fitness.

Dataset	Spearman	p-value
abalone	0.9519	1.10e-10
bank	0.9835	8.29e-15
car-evaluation	0.9246	5.69e-09
diabetes	0.9661	4.91e-12
dry-bean	0.9564	4.62e-11
fire	0.9895	1.46e-16
ml-prove	0.9669	4.00e-12
texture	0.9925	7.19e-18
vehicle	0.9876	6.36e-16
wine-quality-red	0.9628	1.14e-11

4.3 Performance of GGPEnc

We have already shown the results generated by the surrogate have a high accuracy and correlation with the real fitness values. In this section, we evaluate how the encoder impacts the performance of the grammar-based GP. In this section, all experiments represent the average values obtained when running the methods 5 times with different random seeds. All experiments were performed under the same conditions, using the parameters described in Table 1.

We evaluated the evolution of the population using three metrics: maximum fitness, the third quartile, and the median. We applied the t-test to determine if the final results were statistically significant by analyzing the outcomes of the last generation.

Additionally, we measured the evaluation time for both methods. In the GGP, the evaluation time corresponds to the time required to train and test the machine learning model, where fitness is represented by the F1 score obtained on the test dataset. In the GGPEnc, the evaluation time is equivalent to the time needed to sort the population based on the model's predictions. Both times are related to the selection operator, while the other operators are identical for both methods.

Regarding the evolution process, it is desirable for the population to exhibit a certain degree of diversity, meaning that individuals should differ from one another. There are various reasons for this, but from an optimization perspective, it prevents the algorithm from getting stuck in a local optimum. Based on this, we analyzed two aspects of the evolved population: (i) the population's statistics in terms of fitness and (ii) the number of unique individuals.

The statistics of the evolved populations were compared, and the results of the two methods (GGP and GGPEnc) were numerically similar. Therefore, a statistical test was conducted to verify

whether there was any difference between the results obtained by the two methods. We compared only the data from the last generation across all datasets and performed a t-test to compare the maximum value, the third quartile, and the median. The p-values are presented in Table 5. Observed that in 17 out of 30 cases, the null hypothesis (the results of the methods are equal) cannot be rejected with 95% confidence. In the other 13 cases (with values marked in bold), the mean population statistics obtained using the GGPEnc were superior to the GGP in 10 cases. In two of the three cases where the null hypothesis was rejected for the maximum value, the GGP showed superior performance. Nonetheless, in all 13 cases, the absolute difference between the means obtained by the two methods is not significant from a machine learning perspective (the difference between the values is from the second decimal place).

Table 5: P-values from a two-sided t-test accounting for different GP execution. Each column represents the p-value of the t-test comparing a population statistic in the final generation (generation 30).

Dataset	p-value of t-test		
	Maximum	Q(3)	Median
abalone	0.2913	0.0820	0.0409*
bank	0.1187	0.0350*	0.0412*
car-evaluation	0.5875	0.3400	0.1683
diabetes	0.0428	0.5199	0.0018*
dry-bean	0.1347*	0.0397*	0.0331*
fire	0.2443	0.0898	0.0554
ml-prove	0.4826	0.0152*	0.0218*
texture	0.0481	0.1285	0.8171
vehicle	0.0272	0.5645	0.5598
wine-quality-red	0.2861	0.1628	*0.0308

We also monitored the evolution process across all generations, analyzing the population's median, as it is not sensitive to outliers. Figure 5 presents the median. Note the GGPEnc produces a more stable population when compared to the GGP.

Figure 5a shows the median of the population evolved with GGPEnc on the *wine-quality-red* dataset, in contrast to the GGP. For this dataset, the model was trained for 100 epochs. Note that the difference in performance between the two methods is greater in this dataset than when compared to the difference obtained in the other datasets presented, as can be seen in Figures 5b-5d. The GGPEnc produced a population with a median of 0.0187, or 3% higher than the GGP. However, the subsequent statistical comparison indicates that this difference was not statistically significant.

Table 6 presents the median fitness obtained in the last generation (generation 30). In the 'GGPEnc' and 'GGP' columns, the values obtained by the two methods are similar. We applied a t-test to determine whether the differences between the means are statistically significant. As shown in the 'p-value' column, with a 95% confidence level, the null hypothesis can be rejected for the abalone, bank, diabetes, dry-bean, ml-prove, and wine-quality-red datasets. In these datasets, the model's performance was superior to that of the GGP. This suggests that the proposed approach, GGPEnc,

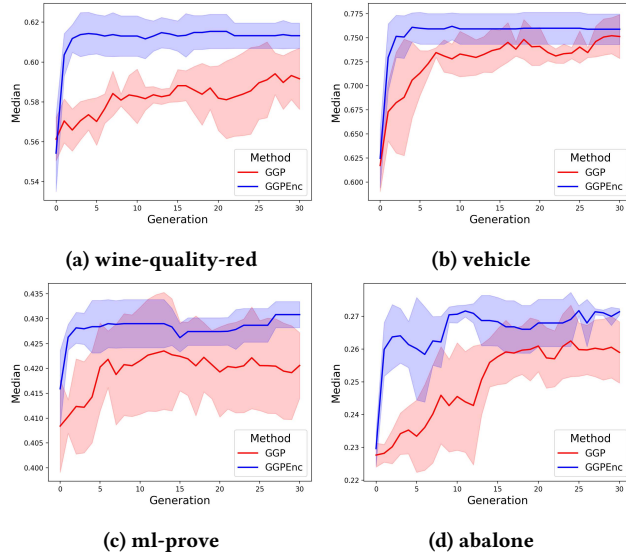


Figure 5: Median fitness of each generation across 4 different datasets. The bold line represents the mean value over five iterations, while the lighter region indicates the standard deviation. Blue represents the surrogate model, and red represents the standard GGP.

is comparable to or better than the GGP, which directly evaluates fitness.

Table 6: Comparison of the population’s median values between GGP and GGPenc. The values represent the mean of five repetitions. The p-value is the result of a two-sided t-test.

Dataset	GGPenc	GGP	p-value
abalone	0.2713	0.2590	0.0409
bank	0.8333	0.8032	0.0412
car-evaluation	0.9192	0.8970	0.1683
diabetes	0.7639	0.7459	0.0018
dry-bean	0.9259	0.9181	0.0331
fire	0.9506	0.9118	0.0554
ml-prove	0.4308	0.4206	0.0218
texture	0.9790	0.9776	0.8171
vehicle	0.7589	0.7513	0.5598
wine-quality-red	0.6132	0.5916	0.0308

Finally, we look at the execution time required by GGP and GGPenc, reported in Figure 6. For GGP, this time includes the fitness calculation (training and testing), whereas for the surrogate method, it involves sorting the population based on the model’s output. Observe that the evaluation time in GGPenc is significantly lower than in GGP. GGPenc had a nearly constant evaluation time of 3 seconds with a standard deviation of 0.1 seconds, regardless of the dataset.

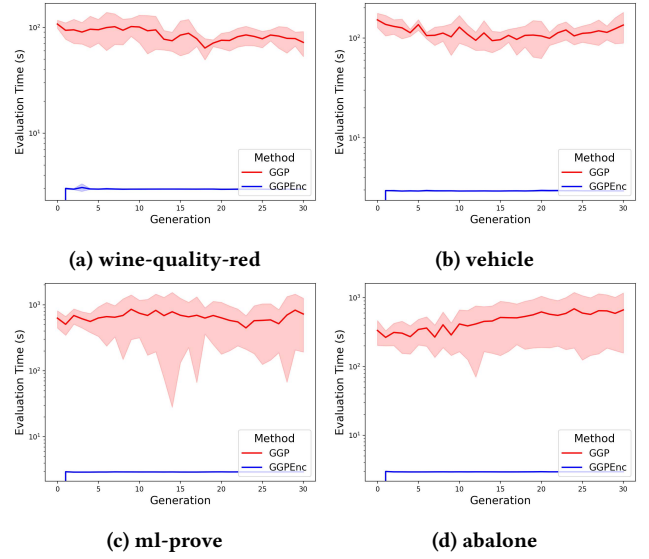


Figure 6: Evaluation time in seconds. The GGP (using fitness evaluation) represents the time required to train and test the model to obtain its fitness value. In the GGPenc, it corresponds to the time needed to sort the population based on the model’s output.

5 Conclusions and Future Work

This paper analyzed the impact of replacing the fitness evaluation of a grammar-based GP with a transformer encoder and evaluated its effect on the GGP evolution compared to the stand-alone method. We first trained the encoder model to classify, given a pair of ML pipelines, which one had better performance between the two. Then, we used this algorithm to rank the individuals during the GGP selection process.

The results indicated that the population evolved using the surrogate has a fitness value equal to (and in some cases superior to) the stand-alone GGP. The main advantage of using the surrogate lies in the time required for population evaluation, which can be up to 285 times faster than GGP.

An important observation is that only one model was trained for 100 epochs for a specific dataset. This model was then transferred to other datasets and fine-tuned for other 30 epochs, achieving similar performance to the GGP with full fitness calculation. On the other hand, the dataset used was significantly large, so a future study will focus on analyzing the effect of reducing the dataset size—that is, verifying the best trade-off between sample size and model accuracy. Adapting the model for other cost-intensive evolutionary problems and analyzing the generalization of this approach in new contexts is also subject of future work.

Our code and experiments are available at: <https://github.com/mct96/gecco-2025>

References

- [1] [n. d.]. Kaggle. <https://www.kaggle.com>.
- [2] 2023. GPT-4 Technical Report. (3 2023).
- [3] Shun-ichi Amari. 1993. Backpropagation and stochastic gradient descent method. *Neurocomputing* 5, 4-5 (1993), 185–196.

- [4] Pierre Baldi and Peter J Sadowski. 2013. Understanding dropout. *Advances in neural information processing systems* 26 (2013).
- [5] Wolfgang Banzhaf, Peter Nordin, Robert E Keller, and Frank D Francone. 1998. *Genetic programming: an introduction: on the automatic evolution of computer programs and its applications*. Morgan Kaufmann Publishers Inc.
- [6] Nils Bjorck, Carla P Gomes, Bart Selman, and Kilian Q Weinberger. 2018. Understanding batch normalization. *Advances in neural information processing systems* 31 (2018).
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. (10 2018).
- [8] Matthias Feurer, Aaron Klein, Katharina Eggenberger, Jost Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and robust automated machine learning. In *Advances in neural information processing systems*. 2962–2970.
- [9] Yang Gu, Hengyu You, Jian Cao, and Muran Yu. 2024. Large Language Models for Constructing and Optimizing Machine Learning Workflows: A Survey. (11 2024).
- [10] Hao Hao, Jinyuan Zhang, Xiaofen Lu, and Aimin Zhou. 2020. Binary relation learning and classifying for preselection in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation* 24, 6 (2020), 1125–1139.
- [11] Hao Hao, Xiaoqun Zhang, and Aimin Zhou. 2024. Large Language Models as Surrogate Models in Evolutionary Algorithms: A Preliminary Study. (6 2024). <https://doi.org/10.1016/j.swevo.2024.101741>
- [12] Xin He, Kaiyong Zhao, and Xiaowen Chu. 2021. AutoML: A survey of the state-of-the-art. *Knowledge-based systems* 212 (2021), 106622.
- [13] Markelle Kelly, Rachel Longjohn, and Kolby Nottingham. 2025. The UCI Machine Learning Repository. <https://archive.ics.uci.edu>.
- [14] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. 2022. Transformers in Vision: A Survey. *Comput. Surveys* 54 (1 2022), 1–41. Issue 10s. <https://doi.org/10.1145/3505244>
- [15] Shengcai Liu, Caishun Chen, Xinghua Qu, Ke Tang, and Yew-Soon Ong. 2023. Large Language Models as Evolutionary Optimizers. (10 2023).
- [16] Yinhan Liu, Mylène Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. (7 2019).
- [17] Robert I McKay, Nguyen Xuan Hoai, Peter Alexander Whigham, Yin Shan, and Michael O’neill. 2010. Grammar-based genetic programming: a survey. *Genetic Programming and Evolvable Machines* 11 (2010), 365–396.
- [18] Randal S. Olson and Jason H. Moore. 2016. TPOT: A Tree-based Pipeline Optimization Tool for Automating Machine Learning. In *Proceedings of the Workshop on Automatic Machine Learning (Proceedings of Machine Learning Research, Vol. 64)*, Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren (Eds.). PMLR, New York, New York, USA, 66–74. https://proceedings.mlr.press/v64/olson_tpot_2016.html
- [19] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [20] Cristiano G. Pimenta, Alex G. C. de Sá, Gabriela Ochoa, and Gisele L. Pappa. 2020. Fitness Landscape Analysis of Automated Machine Learning Search Spaces. 114–130. https://doi.org/10.1007/978-3-030-43680-3_8
- [21] Anish Shah, Eashan Kadam, Hena Shah, Sameer Shinde, and Sandip Shingade. 2016. Deep residual networks with exponential linear unit. In *Proceedings of the third international symposium on computer vision and the internet*. 59–65.
- [22] Xinying Song, Alex Salcianu, Yang Song, Dave Dopson, and Denny Zhou. 2020. Fast WordPiece Tokenization. (12 2020).
- [23] Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. 2019. How to fine-tune bert for text classification?. In *Chinese computational linguistics: 18th China national conference, CCL 2019, Kunming, China, October 18–20, 2019, proceedings* 18. Springer, 194–206.
- [24] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2013. OpenML: Networked Science in Machine Learning. *SIGKDD Explorations* 15, 2 (2013), 49–60. <https://doi.org/10.1145/2641190.2641198>
- [25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [26] Amala Mary Vincent and P Jidesh. 2023. An improved hyperparameter optimization framework for AutoML systems using evolutionary algorithms. *Scientific Reports* 13, 1 (2023), 4737.
- [27] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. 2022. Transformers in Time Series: A Survey. (2 2022).
- [28] Ian H. Witten and Eibe Frank. 2005. Data mining - practical machine learning tools and techniques, Second Edition. In *The Morgan Kaufmann series in data management systems*.
- [29] Tim Z. Xiao, Robert Bamler, Bernhard Schölkopf, and Weiyang Liu. 2024. Verbalized Machine Learning: Revisiting Machine Learning with Language Models. (6 2024).
- [30] Jinglue Xu, Jialong Li, Zhen Liu, Nagar Anthel Venkatesh Suryanarayanan, Guoyuan Zhou, Jia Guo, Hitoshi Iba, and Kenji Tei. 2024. Large Language Models Synergize with Automated Machine Learning. (5 2024).
- [31] Shujian Zhang, Chengyue Gong, Lemeng Wu, Xingchao Liu, and Mingyuan Zhou. 2023. AutoML-GPT: Automatic Machine Learning with GPT. (5 2023).
- [32] Zijun Zhang. 2018. Improved adam optimizer for deep neural networks. In *2018 IEEE/ACM 26th international symposium on quality of service (IWQoS)*. Ieee, 1–2.